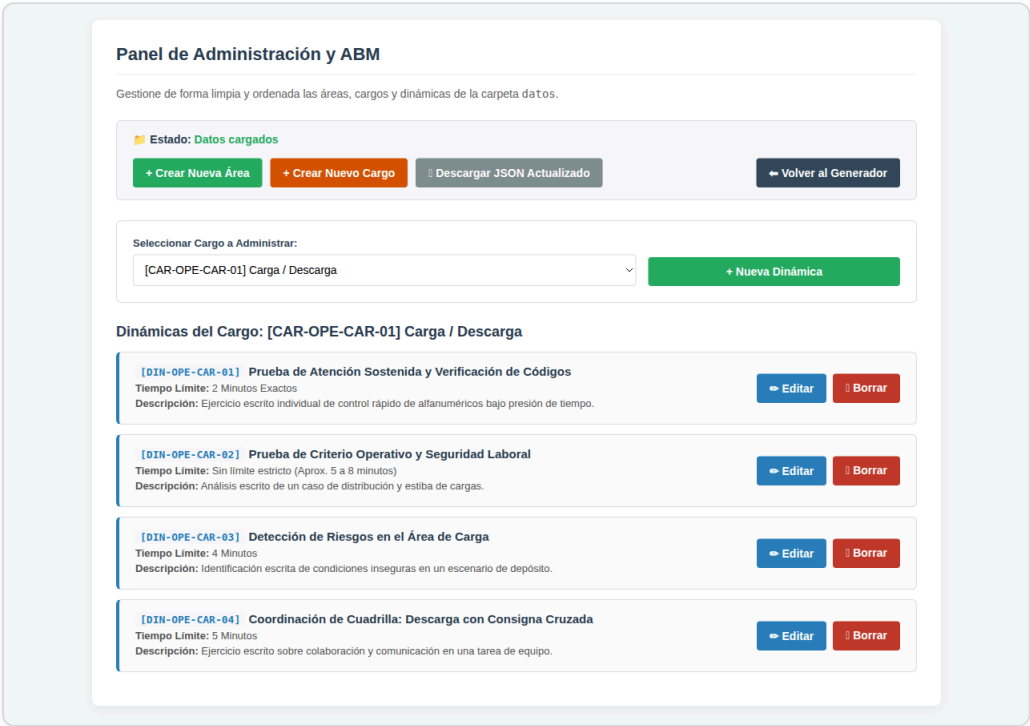


Manual Técnico

Gestor de Pruebas para Llamados

Funcionamiento, datos, instalación y mantenimiento



Versión: 1.0

Fecha: Octubre 2026

Repositorio: FABIOR1981/gestor_pruebas_llamados

Índice

1. Descripción general	3
2. Estructura del proyecto	3
3. Instalación y ejecución	4
3.1 Requisito: servidor HTTP	4
3.2 Ejecución local	4
3.3 Publicación (Netlify u otro hosting estático)	4
4. Modelo de datos	4
4.1 Relaciones	4
4.2 Códigos	5
4.3 areas.json	5
4.4 cargos.json	5
4.5 dinamicas.json	5
5. Funcionamiento del generador (script.js)	6
5.1 Cálculo de tiempos	7
5.2 Cálculo de puntajes	7
5.3 Impresión	7
6. Panel ABM (admin.js)	7
6.1 Procedimiento para publicar cambios	7
6.2 Cambios que requieren editar los JSON a mano	7
7. Asistente de IA (bases_ia.js)	8
7.1 Flujo	8
7.2 Incorporar la respuesta de la IA	8
7.3 Limitaciones conocidas	8
8. Solución de problemas técnicos	9
9. Catálogo incluido	9

1. Descripción general

Aplicación web **estática** (HTML5, CSS3 y JavaScript sin frameworks) que genera paquetes de evaluación psicotécnica: informe para el evaluador, matriz de puntaje por postulante y hojas de trabajo imprimibles en A4. Incluye un panel ABM para mantener el catálogo y un asistente que arma un *prompt* para IA a partir de las bases del llamado.

- No hay backend, base de datos ni autenticación.
- Los datos se leen de archivos JSON en la carpeta `datos/`.
- Todo el procesamiento ocurre en el navegador del usuario.
- Única dependencia externa: **Mammoth.js 1.6.0** (lectura de .docx), que se carga desde cdnjs solo al usar el asistente de IA.

Para el uso de las pantallas, ver el **Manual de Usuario** (`docs/MANUAL_USUARIO.md`).

2. Estructura del proyecto

Archivo	Función
<code>index.html</code>	Pantalla principal (Generador de Evaluaciones) y área imprimible.
<code>script.js</code>	Lógica del generador: carga de datos, filtrado de dinámicas, cálculo de tiempos y puntajes, armado del informe y de las hojas del postulante. Estado global en <code>baseDatosGlobal</code> .
<code>style.css</code>	Estilos de la pantalla principal y reglas de impresión (<code>@page A4</code> , saltos de página).
<code>admin.html</code>	Panel de Administración (ABM) con los modales de área, cargo y dinámica.
<code>admin.js</code>	Lógica del ABM: altas, edición, bajas, generación de códigos y exportación.
<code>admin.css</code>	Estilos del panel ABM.
<code>bases_ia.js</code>	Módulo autónomo del asistente "Directrices desde bases (IA)". Inyecta su propio botón, modal y estilos.
<code>datos/datos.js</code>	Única capa de acceso a datos (objeto <code>Datos</code> con <code>cargar()</code> y <code>exportar()</code>).
<code>datos/areas.json</code>	Catálogo de áreas.
<code>datos/cargos.json</code>	Catálogo de cargos y sus competencias.
<code>datos/dinamicas.json</code>	Catálogo de dinámicas.
<code>utilidades.js</code>	Función <code>esc()</code> para escapar HTML (actualmente no se incluye en las páginas).
<code>docs/</code>	Manuales e imágenes.

3. Instalación y ejecución

3.1 Requisito: servidor HTTP

`datos/datos.js` obtiene los JSON con `fetch()`. Los navegadores bloquean `fetch` sobre `file://`, por lo que la aplicación **debe servirse por HTTP**. Si se abre `index.html` con doble clic aparece la alerta "Error de conexión. Asegúrate de correr la app en Netlify o un servidor local."

3.2 Ejecución local

Desde la raíz del proyecto, cualquiera de estas opciones:

```
python -m http.server 8080
npx serve .
```

Luego abrir `http://localhost:8080` (generador) o `http://localhost:8080/admin.html` (ABM).

3.3 Publicación (Netlify u otro hosting estático)

No requiere *build*: se publica la carpeta raíz tal cual (publish directory = `/`, sin comando de build). Cualquier hosting de archivos estáticos sirve (Netlify, GitHub Pages, IIS, Apache, Nginx).

4. Modelo de datos

4.1 Relaciones

<code>areas.json</code>	<code>cargos.json</code>	<code>dinamicas.json</code>
"FIN"	"CAR-FIN-CON-01"	"cargo_id": "CAR-FIN-CON-01"
(sigla)	area = "Finanzas" (nombre)	id = "DIN-FIN-CON-01"

- Un **cargo** pertenece a un área, pero guarda el **nombre** del área (campo `area`), no la sigla. La sigla queda implícita en el código del cargo.
- Una **dinámica** se vincula al cargo por `cargo_id`, que debe coincidir **exactamente** con la clave del cargo.

4.2 Códigos

Elemento	Formato	Generación
Área	AAA	Sigla de 3 letras ingresada por el usuario (se pasa a mayúsculas, no se permiten duplicados).
Cargo	CAR-<área>-<esp>-<NN>	<esp> = hasta 3 letras ingresadas, completadas con X . NN = mayor número existente con ese prefijo + 1.
Dinámica	DIN-<área>-<esp>-<NN>	Prefijo tomado del código del cargo; NN = mayor existente + 1.

4.3 areas.json

Objeto cuya clave es la sigla:

```
{
  "FIN": { "nombre": "Finanzas" }
}
```

4.4 cargos.json

Objeto cuya clave es el código del cargo:

```
{
  "CAR-FIN-CON-01": {
    "nombre": "Contador",
    "area": "Finanzas",
    "competencias": [
      { "comp": "Pensamiento Analítico", "desc": "Analiza con lógica y detecta incoherencias." }
    ]
  }
}
```

Las **competencias** definen las columnas de la tabla de puntaje. Los cargos creados desde el ABM reciben dos competencias de ejemplo que deben reemplazarse editando este archivo.

4.5 dinamicas.json

Arreglo de objetos:

Campo	Uso
<code>id</code>	Código único de la dinámica.
<code>cargo_id</code>	Código del cargo al que pertenece.
<code>titulo</code>	Título.
<code>desc</code>	Descripción breve (informe del evaluador).
<code>tiempo_limite</code>	Texto libre; los números se usan para el tiempo total.
<code>hoja_postulante</code>	Consigna para el candidato. Admite HTML; se muestra con <code>white-space: pre-line</code> (respeta <code>\n</code>).
<code>caso_o_consigna</code>	Objetivo de la prueba (informe del evaluador).
<code>guia_evaluacion</code>	Guía de observación; los <code>\n</code> se convierten en <code>
</code> .
<code>respuesta_esperada</code>	Corrector; los <code>\n</code> se convierten en <code>
</code> .

Renglones de respuesta: al final de `hoja_postulante` se agregan elementos `<div class=' renglon-respuesta'></div>`, uno por renglón. El ABM los separa al editar (campo *Renglones para escribir*, 0 a 30) y los vuelve a componer al guardar.

```
{
  "id": "DIN-FIN-CON-01",
  "cargo_id": "CAR-FIN-CON-01",
  "titulo": "...",
  "desc": "...",
  "tiempo_limite": "8 Minutos",
  "hoja_postulante": "Consigna...\n\n<div class=' renglon-respuesta'></div><div
class=' renglon-respuesta'></div>",
  "caso_o_consigna": "...",
  "guia_evaluacion": "• Éxito: ...\n• Alerta: ...",
  "respuesta_esperada": "..."
}
```

5. Funcionamiento del generador (`script.js`)

1. Al cargar la página, `cargarDatos()` llama a `Datos.cargar()`, que lee los tres JSON en paralelo y los guarda en `baseDatosGlobal = { areas, cargos, dinamicas }`.
2. `inicializarSelectCargos()` llena el selector de cargos; `actualizarDinamicas()` muestra como casillas las dinámicas con `cargo_id` igual al cargo elegido.
3. Cualquier cambio de cargo o de selección llama a `limpiarResultados()`, que borra el informe para evitar mezclar datos.
4. `generarEvaluacion()` arma el informe, la leyenda de competencias, una tabla por postulante y una hoja por dinámica y por postulante.

5.1 Cálculo de tiempos

`minutosDe(din)` extrae todos los números de `tiempo_limite` y toma el mayor (p. ej. "Aprox. 5 a 8 minutos" → 8; "2 Minutos de Simulación" → 2). El total es la suma de las dinámicas seleccionadas. Textos sin números cuentan 0.

5.2 Cálculo de puntajes

Con c competencias y d dinámicas, cada celda se puntúa de 0 a 5:

Total	Máximo
Por dinámica (fila)	$c \times 5$
Por competencia (columna)	$d \times 5$
General	$c \times d \times 5$

5.3 Impresión

- `@page { size: A4; margin: 15mm } y print-color-adjust: exact .`
- `.no-print` (panel de control) se oculta al imprimir.
- Cada `.candidate-sheet` (hoja del postulante) fuerza un salto de página antes.
- El interruptor *Guía y corrector* agrega la clase `ocultar-guia` al área imprimible, que oculta los bloques `.bloque-guia` (guía de observación y corrector) en pantalla y en papel.

6. Panel ABM (`admin.js`)

- Trabaja sobre la copia en memoria `baseDatosGlobal`. **No escribe en el servidor:** los cambios se pierden al recargar si no se exportan.
- Altas: áreas (`guardarArea`), cargos (`guardarCargo`) y dinámicas (`guardarDinamica`), con códigos autogenerados (sección 4.2).
- Edición y baja: solo para dinámicas (`editarDinamica` , `eliminarDinamica`). Al editar, el código no cambia aunque se cambie el cargo.
- Exportación: `Datos.exportar()` descarga `areas.json` , `cargos.json` y `dinamicas.json` (indentados a 4 espacios).

6.1 Procedimiento para publicar cambios

1. El usuario hace los cambios en el ABM y presiona **Descargar JSON Actualizado**.
2. Reemplazar los tres archivos en `datos/`.
3. Validar que los JSON sean correctos (abrir la app localmente o usar un validador JSON).
4. Hacer *commit* y publicar (por ejemplo, Netlify toma los cambios del repositorio).

6.2 Cambios que requieren editar los JSON a mano

- Definir o modificar las **competencias** de un cargo.
- Renombrar o eliminar **áreas** o **cargos**.

- Corregir el `cargo_id` de una dinámica.

7. Asistente de IA (`bases_ia.js`)

Módulo autónomo (función autoejecutable) que en `DOMContentLoaded` inyecta su botón, su modal y sus estilos en `index.html` . Reutiliza `baseDatosGlobal` para conocer áreas, cargos y dinámicas existentes.

7.1 Flujo

1. Carga Mammoth.js desde `cdnjs.cloudflare.com` (solo la primera vez) y extrae el texto del .docx con `mammoth.extractRawText` . El archivo no sale del navegador.
2. **Competencias:** marca las casillas cuyo patrón (expresión regular del objeto `COMPETENCIAS`) aparece en el texto (12 competencias predefinidas).
3. **Tareas:** toma hasta 12 líneas que estén dentro de secciones con títulos *Funciones / Tareas / Cometidos / Responsabilidades*.
4. **Cargo y área:** busca "cargo de ..." / "área de ..." o nombres ya existentes en el catálogo. Compara con el catálogo (ignorando mayúsculas y tildes) y ofrece coincidencias similares.
5. **Anonimización** (`anonimizar`):

Patrón	Reemplazo
Correos electrónicos	[EMAIL]
Documentos tipo 1.234.567-8	[DOC]
Fechas dd/mm/aaaa y variantes	[FECHA]
Montos con \$, U\$S , USD , UYU	[MONTO]
"llamado / concurso / licitación" + número	[LLAMADO]
Términos indicados por el usuario	[X]

1. **Códigos** (`generarIds`): si el área o el cargo no existen calcula códigos nuevos sin colisiones, y reserva 4 códigos de dinámica consecutivos.
2. Arma el *prompt*: pide 4 dinámicas en bloques JSON listos para pegar (área y cargo solo si son nuevos) e incluye las dinámicas ya existentes del cargo para evitar repeticiones.

7.2 Incorporar la respuesta de la IA

1. Pegar cada bloque en el archivo que indica (`areas.json` y `cargos.json` son objetos; `dinamicas.json` es un arreglo). Cuidar las comas entre elementos.
2. Revisar el contenido (consignas, respuestas esperadas, competencias del cargo).
3. Validar los JSON, probar localmente y publicar.

7.3 Limitaciones conocidas

- La detección de cargo puede capturar texto de más (p. ej. "Tesorero del área de Finanzas").
- Solo acepta `.docx` ; requiere acceso a `cdnjs`.

8. Solución de problemas técnicos

Síntoma	Causa	Solución
Alerta "Error de conexión..." / Estado "Error al leer los datos".	Página abierta como <code>file://</code> , JSON faltante o con error de sintaxis.	Servir por HTTP; revisar la consola del navegador (F12) y validar los JSON.
Selector de cargos vacío.	Error de sintaxis en <code>cargos.json</code> .	Validar el JSON (comas, comillas, llaves).
"No hay dinámicas para este cargo".	Ninguna dinámica tiene <code>cargo_id</code> igual a la clave del cargo.	Corregir <code>cargo_id</code> o crear dinámicas.
Cambios del ABM perdidos.	Se recargó sin exportar.	Exportar siempre antes de salir.
Solo se descarga un JSON.	El navegador bloquea descargas múltiples.	Permitir descargas múltiples para el sitio.
"No se pudo cargar el lector de .docx".	Sin acceso a cdnjs.	Verificar conexión o proxy; opcionalmente alojar Mammoth localmente y cambiar <code>MAMMOTH_URL</code> en <code>bases_ia.js</code> .
Impresión sin colores.	<i>Gráficos de fondo</i> desactivado.	Activarlo en el diálogo de impresión.

9. Catálogo incluido

Cargo	Código	Competencias	Dinámicas
Carga / Descarga	CAR-OPE-CAR-01	Actitud hacia la Normativa · Resistencia / Dinamismo · Trabajo en Equipo	4
Administrativo	CAR-ADM-GEN-01	Atención al Detalle y Orden · Gestión del Tiempo · Comunicación Efectiva	4
Encargado	CAR-SUP-ENC-01	Liderazgo Práctico · Toma de Decisiones · Orientación a Resultados	4
Jefe	CAR-SUP-JEF-01	Visión Estratégica · Negociación e Influencia · Gestión de Crisis	4
Contador	CAR-FIN-CON-01	Método y Criterio Profesional · Pensamiento Analítico · Integridad y Confidencialidad	4